BudPay: The Fair Pay Helper

Project plan for the PayPal AI Hackathon • Written Nov 4, 2026 • "BudPay" is a working name and can change

1. The story in one minute

Imagine a tailor named Tunde. A customer asks him to sew outfits for a wedding. Tunde cannot do it alone, so:

- His **apprentice** sews the clothes.
- A **seamstress** does the fancy hand finishing.
- A **rider** delivers the outfits.

The customer pays Tunde **one big payment**. Now Tunde has a hard job: sharing the money. Who worked the most? Who should get what? It feels awkward, he might forget someone, and doing the math by hand is slow.

BudPay is a helper app that makes this easy.

1. Tunde tells the helper who did what, in his own words.
2. The helper suggests how to share the money.
3. Tunde looks at it, changes anything he wants, and decides.
4. He presses **Send**, and PayPal sends everyone their money at once, each with a thank-you note.

In one sentence: PayPal moves the money. BudPay helps the boss decide what is fair.

2. Who is it for?

- **The boss (the tailor).** He is the person who uses the app. He is in charge.
- **The crew (apprentice, seamstress, rider).** They only receive money and a thank-you note.
- **The judges.** They will watch our demo and decide if the idea is good.

3. What problem are we fixing?

- Sharing money is **awkward**.
- The boss may **forget** who did what.

- The crew has to **guess** what they will get.
- Doing the math by hand is **slow** and easy to get wrong.

An important rule: In real life, the tailor gives his crew money as a thank-you, and nobody can force him. So our helper never forces anything. It only **suggests**. The boss always has the last word.

4. What are we actually building?

One website with six steps:

1. **The customer pays.** We use PayPal's pretend-money playground (more on that below) so nobody loses real money.
2. **The boss tells the story.** He types (or speaks) something like: *"Tola sewed for 8 hours. Bisi did the finishing for 5 hours. Kunle did the delivery, about 2 hours."*
3. **The helper reads the story.** It turns it into a neat list: who, what job, how many hours. If something is missing, it asks.
4. **The boss picks his own share.** He moves a slider. Whatever is left is the **crew pile** of money.
5. **The helper suggests a split of the crew pile.** Each person gets a short reason, like "sewed the most hours." The boss can change any number he wants.
6. **The boss presses Send.** PayPal sends the payments to everybody at the same time, each with a personal thank-you note.

A worked example

- The customer pays **$200**.
- Cloth, thread and other costs are **$40**. That money is not anyone's earnings, so it comes off first. **$160 is left.**
- Tunde decides to keep **$90** for himself. The **crew pile is $70.**

Person	Job	Hours × job weight	Points	Their share of the $70
Apprentice	Sewing	8 × 1.5	12	**$35.00**
Seamstress	Finishing	5 × 2.0	10	**$29.17**
Rider	Delivery	2 × 1.0	2	**$5.83**

Total: 35.00 + 29.17 + 5.83 = **$70.00**. Nothing is lost and nothing is made up.

5. How does the sharing work? (The pizza idea)

Think of the crew pile as a pizza. Everyone gets slices, and the more they helped, the more slices they get.

1. **Points = hours × job weight.** The "job weight" is how much skill the work needs. Delivery is 1.0, sewing is 1.5, and careful finishing is 2.0. The helper suggests these weights, and the boss can change them.
2. **Add up everyone's points.**
3. **Each person's slice = their points ÷ everyone's points.** That slice of the crew pile is their money.
4. **The boss can change anything.** He might add a bonus because the job was a rush, or give a little more to a new apprentice who is learning.

Pennies rule: We work in whole cents so the numbers always add up exactly. If one cent is left over, it goes to the person with the biggest share.

6. What the helper does, and what it never does

The helper DOES:

- Read the boss's story and turn it into a list.
- Ask a question if something is missing.
- Suggest a fair split and explain it in plain words.
- Write a short thank-you note for each person.
- Warn the boss if something looks strange (for example, one person asking for 80%).
- *(Bonus)* Learn the boss's habits from past jobs, like "you usually give the apprentice about 15% on sewing jobs," and use that as the starting suggestion.

The helper NEVER:

- Touches the boss's own share.
- Sends money by itself. Only the boss pressing **Send** does that.
- Makes the final decision.
- Forces the crew or the boss to follow its numbers.

7. What if someone cheats?

If crew members could write their own hours, someone might say they worked more than they did. In this first version, **only the boss writes the story**, so this cannot happen. Later, if we let the crew log their own work, we would add these safety nets:

- Their notes count as **claims**, and the boss must say yes or no to each one.
- They log **tasks** (like "hemmed 3 trousers") instead of hours, because tasks are easier to check.
- The helper flags times that look too long for the task.
- The helper checks that all hours fit between the start and the delivery date.

8. What we are NOT building (on purpose)

We only have a few days, so we keep it small. We are **not** building:

- Logins for the crew or crew-written work logs (we will mention it as a "next step").
- Photo proof of work.
- Real money. Everything uses PayPal's pretend-money playground.
- Many shops, accounts, tax tools, chat, or a phone app.

A small thing that works perfectly beats a big thing that is half broken.

9. The tools we use

Tool	What it is, in simple words	Its job in our project
Express.js	The kitchen. It does the thinking behind the scenes.	Runs the sharing math and talks to PayPal and the AI.
Next.js	The dining room. It is what people see and click.	The screens: the story box, the review list, the sharing sliders, the Send button.
Gemini AI	A smart reader that understands normal sentences.	Turns the boss's story into a list and writes thank-you notes.
PayPal Orders (sandbox)	The cash register.	Lets the customer pay.

Tool	What it is, in simple words	Its job in our project
PayPal Payouts (sandbox)	A way to send money to many people at once.	Pays the whole crew with one click.
Supabase database	A notebook that remembers.	Saves jobs, people and past payouts.
Vercel	Free places to put our website online.	So judges can open it from a link.

What is "sandbox"? It is PayPal's pretend-money playground. It works like the real PayPal, but the money is not real. We make fake shop, customer and crew accounts and practice there safely.

10. The build plan, step by step

The hackathon closes on **Nov 12**. We plan to **finish on Nov 11** so we have a spare day if something goes wrong.

Milestone 1 (Wed Nov 4 to Thu Nov 5): Wake up the PayPal robot. Set up the PayPal playground accounts (one shop account, one customer, three crew accounts). Create the project folders. Read the official hackathon rules and note every must-have, such as a video or a code link. Send one test payout. *Done when:* a test payout shows as successful and appears in a crew sandbox account. We do this first because it is the riskiest part.

Milestone 2 (Fri Nov 6): Teach the helper to read. Connect Gemini so it turns a story into a clean list. *Done when:* we try 5 different sample stories and the list is right for at least 4 of them, and the helper asks a question when information is missing.

Milestone 3 (Sat Nov 7): Do the sharing math. Build the points formula, the crew pile and the pennies rule, with the helper's reasons. *Done when:* the shares always add up to exactly the crew pile, and our automatic math tests pass.

Milestone 4 (Sun Nov 8): Make the screens. Build the story box, the review list and the sharing screen with sliders. *Done when:* a person who has never seen the app can get through all the steps without anyone explaining.

Milestone 5 (Mon Nov 9): Connect the money. Customer checkout comes in, and the Send button makes PayPal pay everyone, each with a thank-you note. *Done when:* one full run from payment to payout works in the playground, and each crew account shows the right amount.

Milestone 6 (Tue Nov 10): Safety checks and habits (bonus). Add the "this looks odd" warning and the learning of the boss's habits using a few practice jobs. *Done when:* after 3 practice jobs the helper's first suggestion matches the boss's usual pattern, and the warning shows up for a strange number. If we are behind, we skip this one.

Milestone 7 (Wed Nov 11): Polish and show. Put the app online, record a demo video of about 2 to 3 minutes, and write the project description. *Done when:* the link works from a brand new browser and the video is saved.

Thu Nov 12: Submit early. This is our safety day. We do not plan any new work here.

Day by day: what we do each day

Here is exactly what to do each day. Every day ends with a small test, so we know the day is really finished. **Every evening:** save the code to GitHub and write one line about what is done and what is stuck.

Day 1 · Wed Nov 4: Set up the kitchen and the dining room

- Make two folders: one for the backend (the kitchen, Express.js) and one for the frontend (the dining room, Next.js).
- Start both and make them say hello to each other.
- Put the code on GitHub. Keep secret keys in a private settings file that is never uploaded.
- Make a PayPal developer account and create the pretend accounts: one shop, one customer and three crew members.
- Read the official hackathon rules and write down every must-have.

End of day: both parts run and can talk to each other, and the pretend accounts exist.

Day 2 · Thu Nov 5: Make PayPal work

- Copy the sandbox keys into the private settings file.
- Ask PayPal for an access pass (a token).
- Send one test payout to a crew account.
- Try one test customer payment.
- Set up the notebook (a supabase database) with three lists: jobs, people and payouts.

End of day: the test payout shows as successful in a crew account. If it is stuck, ask for help right away instead of waiting.

Day 3 · Fri Nov 6: Teach the helper to read

- Connect Gemini.
- Write the instructions that turn a story into a list: who, what job and how many hours.
- Make 5 practice stories and check the lists.
- Teach the helper to ask a question when something is missing.

End of day: the list is right for at least 4 of the 5 stories.

Day 4 · Sat Nov 7: Do the sharing math

- Take off the costs, then the boss's share, to find the crew pile.
- Add the points formula, the slices and the pennies rule.
- Have the helper write a short reason for each person.
- Add the money guardrails in code: the AI never sees the boss's share, the total must match to the cent, and every amount is checked (see section 16).
- Write tests that check the math, plus one test for each guardrail.

End of day: the shares always add up to exactly the crew pile, and all tests pass.

Day 5 · Sun Nov 8: Make the screens

- Build the story box, the review list and the sharing screen with sliders and a bonus box.
- Add the Send button (it does not send money yet).
- Connect the screens to the backend.
- Ask a friend to try it without any help.

End of day: someone who has never seen the app can finish all the steps.

Day 6 · Mon Nov 9: Connect the money

- Add the page where the customer pays.
- Make the Send button pay everyone at once, each with a thank-you note.
- Save every payout in the notebook.
- Build the receipt screen: after Send, ask PayPal how each person's payment is doing and show "Paid," "Waiting" or "Problem" next to each name. Only show "Paid" when PayPal says so, not when we only sent the request.
- Check that the amounts PayPal paid match our plan and add up to the crew pile. If not, show a warning.
- Add the safety locks in code: a job can only be paid after the boss approves it, a job can only be paid once, and only people on the crew list can be paid.

- Make the app refuse to start unless it is using PayPal's pretend-money (sandbox) address.
- Run the whole thing from start to finish, and log in to the three crew sandbox accounts to see the money arrive.

End of day: one full run works, each crew account shows the right amount, pressing Send twice does not pay twice, and the receipt screen shows "Paid" next to every person.

Day 7 · Tue Nov 10: Safety checks, habits and bug fixing

- Add the "this looks odd" warning.
- Teach the helper the boss's habits from 3 practice jobs (bonus).
- Fix the bugs found in yesterday's full run.
- If we are behind, skip the bonus parts and only fix bugs.

End of day: 3 full runs in a row with no problems.

Day 8 · Wed Nov 11: Polish and show

- Put the app online (Render and Vercel).
- Open the link from a brand new browser to be sure it works.
- Record the 2 to 3 minute demo video using the demo story.
- Write the project description and tick off the rules list from Day 1.
- Check that no secret keys are in the public code.

End of day: the link works, the video is saved and the description is ready.

Day 9 · Thu Nov 12: Submit

- Submit in the morning. No new features today. This is our spare day if something goes wrong.

11. What does "done" mean for the whole project?

The project is done when **all** of these are true:

- A person can go from the boss's story to the crew getting paid, in under 3 minutes.
- At least **3 pretend accounts** receive their money.
- The shares always add up exactly.
- The boss can change any number, and the helper never touches his own share.
- Nothing is sent until the boss presses **Send**.

- Thank-you notes arrive with the payments.
- The website opens from a public link.
- No secret passwords or keys are in the public code.
- The demo video and project description are finished.

12. If we run out of time, what do we cut?

We cut things in this order:

1. Learning the boss's habits.
2. The "this looks odd" warning.
3. AI-written thank-you notes (we use a simple template instead).
4. Fancy styling.

We never cut: the customer paying, the sharing math, and the payout. Without those, there is no project.

13. What could go wrong?

Worry	What we do about it
Sending test payouts does not work in the playground.	We test it on day one. If we cannot fix it, we show the real request and answer clearly and say honestly that it is a demo.
The AI misreads the story.	The boss always checks the list before anything else happens.
The money is off by a penny.	We use whole cents and the pennies rule.
We run late.	We follow the cut list above.
Secret keys leak.	We keep them in a private settings file and never upload it.

14. The demo story (for the video)

1. **The problem (20 seconds):** Tunde got paid, and now sharing is awkward.
2. **The story:** He types who did what.

3. **The helper reads and suggests:** The list appears, then the suggested shares with reasons.
4. **The boss is in charge:** He changes one number and adds a bonus.
5. **Send:** PayPal pays everyone, and we show the thank-you notes arriving.
6. **The closing line:** *"PayPal moves the money. CrewPay helps the boss decide what is fair."*

15. Words to know

- **API:** a waiter that carries orders between two programs.
- **Sandbox:** PayPal's pretend-money playground.
- **Payout:** sending money to a person.
- **Crew pile:** the money left to share with the crew after costs and the boss's own share.
- **Milestone:** a checkpoint on the way, like a level in a game.
- **Done:** a clear test that tells us a step is really finished.

16. Guardrails: the code keeps the rules too

We tell the AI the rules, but we do not trust that alone. The code checks every rule as well. It is like having a locked door and not only a sign that says "Do not enter."

The biggest rule: the AI never decides any money amount. It only reads the story and suggests jobs, hours and reasons. Our own code turns those into dollars, counted in whole cents. So even if the AI makes a mistake, it cannot move money.

The rule	What the AI is told	What the code does, even if the AI gets it wrong
The boss's own share is never touched.	Do not change the boss's share.	The boss's share only comes from his slider. The AI never even sees it.
The shares add up exactly.	Make the numbers add up.	The code counts in whole cents and checks the total. If it is off by even one cent, the Send button is blocked.

The rule	What the AI is told	What the code does, even if the AI gets it wrong
Nothing is sent until the boss says so.	Wait for the boss.	Each job moves through steps: draft, reviewed, approved, sent. Money can only go out when the job is approved. The AI part of the code cannot reach the PayPal part at all.
Only real crew members get paid.	Only pay the people in the story.	Payments go only to people on the boss's saved crew list. The AI can match a name, but it can never add a new email address.
We never pay out more than came in.	Stay inside the budget.	The total payout can never be more than the customer paid minus the costs. Every payment is at least one cent.
Nobody gets paid twice.	(A prompt cannot do this.)	Every job gets its own unique batch ID and is locked after Send. Clicking twice or trying again cannot pay twice.
Odd numbers get a warning.	Warn if something looks strange.	Fixed checks find the odd numbers, like one person getting a huge slice or too many hours for a small job. The AI only writes the warning in friendly words.
Pretend money only.	Use the sandbox.	The app refuses to start if the PayPal address is not the sandbox one.

More guardrails

- **Check everything the AI gives back.** It must have the right shape, sensible hours and a limited number of people. If it fails, we try once more, and then we ask the boss.
- **Treat the boss's story like a note from a stranger.** If it says "give me 100%," nothing changes, because the AI does not set amounts and the code rules still apply.
- **Keep secret keys in the backend only.** The screens never see them.

- **Keep a diary.** Save every suggestion, every change the boss makes and the time he approves.

How we prove the guardrails work. We write one test for each rule:

1. A payout without approval is refused.
2. If the AI returns a boss's share, it is ignored.
3. If the total does not match the crew pile, Send is blocked.
4. A story that says "give me 100%" changes nothing.
5. Pressing Send twice pays only once.
6. A person who is not on the crew list cannot be paid.

We build the money rules and tests on **Day 4**, and the approval steps, the double-send lock and the sandbox check on **Day 6**.